iZprime Framework: A Prime Generation Toolkit

Hazem Mounir

January 12, 2026

Abstract

This paper introduces the *iZprime Framework*, a novel approach to large-scale prime generation that combines a minimal wheel structure with a 2-D segmented sieve design. By leveraging the $i\mathbb{Z}$ set (numbers congruent to $\pm 1 \pmod 6$) and a standardized segmentation modulus, the framework achieves constant auxiliary space and reduced per-segment bit-work. Implemented in the open-source *iZprime* library, this framework offers efficient and scalable prime enumeration suitable for cryptographic applications and computational number theory.

Contents

1 Introduction **2**
1.1 Materials and Methods . . . 2
1.1.1 Dependencies . . . 2
1.1.2 Results Replication . . . 2
1.2 Road Map . . . 2

2 Prime Sieve: Theoretic and Algorithmic Foundations **3**
2.1 Number Theory Basics . . . 3
2.2 Prime Sieving Core Concepts . . . 3
2.2.1 Basic Data Structures: Bitmaps and Integer Arrays . . . 3
2.2.2 Sieve of Eratosthenes (SoE) . . . 3
2.2.3 Important Sieve Optimizations Techniques . . . 4

3 The iZ Structure: Basic Wheel Optimization **5**
3.1 iZ Definitions . . . 6
3.2 The Xp-Identity: Composite Detection within iZ . . . 6
3.3 The Classic Sieve-iZ Algorithm (SiZ) . . . 7

4 The iZm Structure: Extended Wheel Optimization **8**

5 The VX_SEG Module: Large-Scale Hybrid Sieve Tools **10**

6 Random Prime Generation Tools **10**

7 Conclusions **10**

A Algebraic Study on the iZ Function **10**

B Data Structures **10**
B.1 Bitmap Module . . . 10
B.2 Integer Array Module . . . 10

1 Introduction

Large scale prime generation presents an enduring and instructive challenge in computational mathematics and algorithm design, serving as an ideal testbed for practicing techniques like space-time optimization, data structures, and segmentation—core areas of interest in modern algorithm engineering.

1.1 Materials and Methods

We implement the framework in C as the open-source *iZprime* library. Algorithms are presented in language-agnostic pseudocode. For artifact-quality reproducibility, the library provides unit tests, benchmark drivers, and tools to regenerate all figures/tables used in this paper.

1.1.1 Dependencies

Beside the standard C libraries, the following libraries are required:

- **GMP** (GNU Multiple Precision Arithmetic Library) (version 6.2.1) for large integer arithmetic
- **OpenSSL** (version 3.0.7) for cryptographic functions

1.1.2 Results Replication

Benchmarks in this paper were produced under the following environment; full scripts and configuration files in the repository reproduce every figure/table:

- **Hardware:** Apple MacBook Pro (M1, 8 GB unified memory).
- **Environment:** macOS (Apple Silicon) with `Apple clang 17` (`clang-1700.0.13.5`) and flags `-O3 -flto`.

1.2 Road Map

To Aurel Stevens,

2 Prime Sieve: Theoretic and Algorithmic Foundations

This section establishes the mathematics and data layout that make the iZprime Framework practical: a reduced search space, an additive composite-marking identity, and a 2-D segmentation model that yields cache-sized, reusable segments.

2.1 Number Theory Basics

A prime p is a natural number greater than 1 with no divisors other than 1 and p. We write $\mathbb{P}$ for the set of primes and $\pi(n)$ for the prime-counting function, with asymptotic approximation given by the *prime number theorem*:

$$\pi(n) \sim \frac{n}{\log n}. \tag{1}$$

The sum of prime reciprocals diverges to infinity asymptotically as

$$\sum_{p \le n} \frac{1}{p} \sim \log \log n, \tag{2}$$

this result underpins the $O(n \log \log n)$ cost model of Eratosthenes-type sieves.

Finally, we recall Euler's totient function $\varphi(n)$—the count of numbers up to n that are coprime to n—given by

$$\varphi(n) = n \prod_{p \mid n} \Big(1 - \frac{1}{p}\Big), \tag{3}$$

Notably, for a prime p, this function simplifies to $\varphi(p) = p - 1$.

2.2 Prime Sieving Core Concepts

The core concept behind prime sieving is the *fundamental theorem of arithmetic*, which states that every integer greater than 1 is either a prime itself or can be uniquely factored into primes. This theorem implies that to determine the primality of numbers up to a limit n, it suffices to identify and *eliminate* multiples of known primes up to $\sqrt{n}$ to *isolate* the primes. In other words, prime sieving is the systematic marking of composites to reveal the primes.

2.2.1 Basic Data Structures: Bitmaps and Integer Arrays

Prime sieving algorithms typically rely on two basic data structures:

- **Bitmaps:** Compact arrays of bits to represent a range of numbers where each bit represents the primality state (prime or composite) of a corresponding integer. Bitmaps enable efficient storage and fast bitwise operations for marking composites. In our framework, we provide the `BITMAP` type with a suite of operations including initialization, bit setting/clearing, and bit querying. A summary of the `BITMAP` module is provided in Appendix B.1.
- **Integer Arrays:** Dynamic arrays that store the list of identified primes. These arrays facilitate easy appending and retrieval of prime numbers during the sieving process. In our framework, we provide three fixed-size integer arrays: `UI16_ARRAY` for `uint16_t`, `UI32_ARRAY` for `uint32_t`, and `UI64_ARRAY` for `uint64_t`, each with operations for initialization, appending, and accessing elements. A summary of the `INT_ARRAY` module is provided in Appendix B.2.

2.2.2 Sieve of Eratosthenes (SoE)

The archetypal sieve method and the foundation to our optimizations is the *Sieve of Eratosthenes*, attributed to Eratosthenes of Cyrene (circa 200 BCE), remains influential due to its conceptual clarity, ease of implementation, and practical efficiency.

Modern implementations with established speed ups operate as described in 2.2.2.

Pseudocode: `SoE`

Algorithm 1 Sieve of Eratosthenes (SoE)

```
procedure SoE(int : n) → INT_ARRAY
Input: int: n                                          ▷ upper limit
Output: INT_ARRAY: primes                              ▷ array of integers containing primes up to n
                                                       ▷ 1. Initialization
    INT_ARRAY: primes ← {2}                            ▷ initialize primes array with the only even prime
    BITMAP: n_bits ← BITMAP_INIT(n + 1, 1)             ▷ create a bitmap of size n + 1, and all bits set to 1
                                                       ▷ 2. Sieve Process: Marking composites and collecting primes
    int: n_sqrt ← ⌈√n⌉                                 ▷ root limit
    for int: x ← 3 to n with step 2 do                 ▷ iterate through odd numbers up to n
        if BITMAP_GET_BIT(n_bits, x) then              ▷ is prime check
            INT_ARRAY_PUSH(primes, x)                  ▷ append x to primes array
            if x ≤ n_sqrt then                         ▷ if x is a root prime
                BITMAP_CLEAR_STEPS(n_bits, 2x, x^2, n) ▷ mark odd multiples of x starting from x^2
            end if
        end if
    end for
                                                       ▷ 3. Return the result
    return primes
end procedure
```

Space Complexity: Due to the bitmap of size $n+1$, the SoE requires $O(n)$ auxiliary space.

Time Complexity: The outer loop iterates over all odd numbers up to n, and for each prime found, the inner marking loop marks its odd multiples. The bit-work after applying the speed ups is

$$O\left(\frac{n}{2}\sum_{p=3}^{\sqrt{n}}\frac{1}{p}\right)$$

bit-operations, which from [2], grows like $O(n \log\log n)$.

Next, we discuss important optimizations that are widely used to enhance the SoE's scalability and performance.

2.2.3 Important Sieve Optimizations Techniques

Given how space and time complexities limit the scalability of prime sieves, two key optimization techniques have been developed over the years to address these challenges.

Segmentation

The most urgent performance bottleneck in the classic SoE (and all *Solid Sieves*) is the linear space requirement $O(n)$, which becomes prohibitive even for moderate n. To overcome this, *Segmented Sieves* partition the sieve range n into smaller segments of size Δ (often $\sqrt{n}$). Each segment is processed independently, allowing us to process large ranges without exceeding our memory capacity, significantly mitigating the space bottleneck.

The first *Segmented-SoE* (SSoE) is attributed to Bays/Hudson from 1977 [4], then refined over the years by many authors.

The Wheel Concept

In the context of prime sieving, a *wheel* W (often $W_k = \prod_{i=1}^{k} p_i$) is a precomputed structure that filters out entire classes of composite numbers based on their residue classes modulo W.

In modular arithmetic terms, a modulus $m = p_1 p_2$ partitions the integers into m residue classes: $mx + i$ for $i \in \{0, 1, 2, \ldots, m-1\}$. Only partitions where i is coprime to m can host primes, otherwise, it's a multiple of p_1 or p_2 and thus composite.

Examples. For $W_2 = 2 \times 3 = 6$, other than 2 and 3, primes can only be found in the classes $\{1, 5\}$ mod 6. For $W_3 = W_2 \times 5 = 30$, the residue classes to consider for primes are $\{1, 7, 11, 13, 17, 19, 23, 29\}$ mod 30. And so on.

Effect on density. The candidate density under a wheel W is

$$\frac{\varphi(W)}{W} = \prod_{p|W} \Big(1 - \frac{1}{p}\Big),$$

so the wheel removes a constant fraction of non-candidates determined by the included primes.

Advantages. A wheel performs *bulk elimination* of non-candidates before any per-prime work, optimizing both space and time. It reduces the candidate set size, leading to fewer bits to store and process, and decreases the number of composite markings required during sieving.

Limitations. Each additional prime p in the wheel reduces per-prime marking work only by its $1/p$ share, but increases the wheel size and memory traffic by a factor of p. Thus, beyond small bounded wheels, returns diminish and cache behavior degrades.

The wheel concept has a long history in prime sieving, starting from 1960s by Chartres and Singleton, then significant developments by Pritchard and Sorenson, introducing families of wheel based sieves (static and dynamic) and their integration with segmentation.

In our framework, we build upon this rich heritage to design a two-layered wheel structure that balances simplicity, space efficiency, and per-segment performance.

Two-layered wheel design. We deliberately separate between two wheel roles:

1. **Basic static wheel** $6x \pm 1$: excludes multiples of 2 and 3 and reduces the candidate universe to only two symmetric partitions $\pm 1 \pmod 6$, which we call the $i\mathbb{Z}$ set, a threefold reduction to the original $\mathbb{N}$.
2. **Extended wheel** vx_k (product of the first k primes > 3): used as *a segmentation modulus* within the $i\mathbb{Z}$ space to standardize cache-sized tiles and allow pre-sieving of primes dividing vx, further optimizing the candidate set ($i\mathbb{Z}$) by a factor of $\varphi(vx)/vx$.

This combination decouples segmentation from the input size, enabling **constant auxiliary space** $O(1)$ per segment, while also reducing the per-segment work by skipping many small primes (constant-factor improvement).

In the following sections, we formalize this structure and apply it in practice to develop a suite of efficient prime sieving tools, capable of scaling to very large limits.

3 The iZ Structure: Basic Wheel Optimization

A first, low-cost optimization is the basic wheel of $6x \pm 1$, which gets rid of all multiples of 2 and 3, which account for two thirds of $\mathbb{N}$, leaving only two partitions $\pm 1 \pmod 6$. We favor this minimal wheel over larger choices like $30x + i$ because it preserves a tiny branching factor (two lanes instead of eight) while already cutting the candidate set to one third of $\mathbb{N}$.

Now, we need to develop an efficient mechanism to mark composites within this reduced candidate set. Below, we formalize this approach and introduce the Xp-Identity–an efficient additive mechanism for marking composites within the $i\mathbb{Z}$ space.

3.1 iZ Definitions

Definition 3.1 (The $i\mathbb{Z}$ set)**.** The union of two disjoint partitions modulo 6:

$$i\mathbb{Z} \;=\; \{z \in \mathbb{N} : z \equiv \pm 1 \pmod 6\} \;=\; i\mathbb{Z}^- \dot\cup\, i\mathbb{Z}^+, \quad i\mathbb{Z}^- = \{6x-1 : x \in \mathbb{N}\}, \quad i\mathbb{Z}^+ = \{6x+1 : x \in \mathbb{N}\},$$

with cardinality $|i\mathbb{Z}| = \frac{1}{3}|\mathbb{N}|$.

To map between indices x and $6x \pm 1$ values we use the iZ function

Definition 3.2 (The iZ function)**.** Let $\mathbb{I} = \{-1, +1\}$ denote the set of residues for the two $i\mathbb{Z}$ partitions.

$$\boxed{iZ(x \in \mathbb{N},\ i \in \mathbb{I}) = 6x + i}.$$

Here, x is the *index* in a partition and i is the *residue*, indicating which partition (-1 for $i\mathbb{Z}^-$ and $+1$ for $i\mathbb{Z}^+$).

In our code, we provide two implementations of the iZ function with the following signatures:

Listing 1: File: utils.h

```
uint64_t iZ(uint64_t x, int i); // for values < 2^64
void iZ_mpz(mpz_t z, mpz_t x, int i); // for arbitrary values using GMP
```

X-Arrays. In sieving the $i\mathbb{Z}$ set up to a limit n, we maintain two separate bitmaps, one per partition, each of length $x_n = \lceil \frac{n}{6} \rceil$ bits:

- `X5[`x`]` encodes the candidate $6x - 1$ (the $i\mathbb{Z}^-$ lane),
- `X7[`x`]` encodes the candidate $6x + 1$ (the $i\mathbb{Z}^+$ lane).

3.2 The Xp-Identity: Composite Detection within iZ

The Xp-Identity is the engine of our index-based, additive marking. Derived from Lemma A.1

Definition 3.3 (Xp-Identity)**.** Let $p = iZ(x_p, i_p) \in \mathbb{P}$. Then the composite indices of p in the X-Arrays are

$$\begin{aligned} &\texttt{X5:} \quad \left\{\, p\,x_q - i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \text{ maps to the composites of } p \text{ in } i\mathbb{Z}^-, \\ &\texttt{X7:} \quad \left\{\, p\,x_q + i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \text{ maps to the composites of } p \text{ in } i\mathbb{Z}^+. \end{aligned}$$

Here, the lower bound $x_q \geq x_p$ (for the first hit) is to avoid duplicate marking of composites whose smaller cofactor would have been handled earlier.

Example: Let us apply the Xp-Identity to generate the composite indices of the first few primes in the X-arrays and their corresponding values in $i\mathbb{Z}$.

$$5 = iZ(1,-1) \begin{cases} \texttt{X5:} & x \in \{5\cdot 1 + 1 = 6,\ 11,\ {+}{+}5\} \quad iZ(x,-1) \to \{35,\ 65,\ {+}{+}30\} \in i\mathbb{Z}^-, \\ \texttt{X7:} & x \in \{5\cdot 1 - 1 = 4,\ 9,\ {+}{+}5\} \quad iZ(x,+1) \to \{25,\ 55,\ {+}{+}30\} \in i\mathbb{Z}^+. \end{cases}$$

$$7 = iZ(1,+1) \begin{cases} \texttt{X5:} & x \in \{7\cdot 1 - 1 = 6,\ 13,\ {+}{+}7\} \quad iZ(x,-1) \to \{35,\ 77,\ {+}{+}42\} \in i\mathbb{Z}^-, \\ \texttt{X7:} & x \in \{7\cdot 1 + 1 = 8,\ 15,\ {+}{+}7\} \quad iZ(x,+1) \to \{49,\ 91,\ {+}{+}42\} \in i\mathbb{Z}^+. \end{cases}$$

$$11 = iZ(2,-1) \begin{cases} \texttt{X5:} & x \in \{11\cdot 2 + 2 = 24,\ 35,\ {+}{+}11\} \quad iZ(x,-1) \to \{143,\ 209,\ {+}{+}66\} \in i\mathbb{Z}^-, \\ \texttt{X7:} & x \in \{11\cdot 2 - 2 = 20,\ 31,\ {+}{+}11\} \quad iZ(x,+1) \to \{121,\ 187,\ {+}{+}66\} \in i\mathbb{Z}^+. \end{cases}$$

$$13 = iZ(2,+1) \begin{cases} \texttt{X5:} & x \in \{13\cdot 2 - 2 = 24,\ 37,\ {+}{+}13\} \quad iZ(x,-1) \to \{143,\ 221,\ {+}{+}78\} \in i\mathbb{Z}^-, \\ \texttt{X7:} & x \in \{13\cdot 2 + 2 = 28,\ 41,\ {+}{+}13\} \quad iZ(x,+1) \to \{169,\ 247,\ {+}{+}78\} \in i\mathbb{Z}^+. \end{cases}$$

Sieve Logic. By marking all composite indices of root primes $5 \leq p \leq \sqrt{n}$ in the X-arrays, we can collect the unmarked indices as the primes greater than 3 and up to n.

Bit-work consequence. Using two bitmaps, each of length $\lceil n/6 \rceil$, and strictly additive stepping, a solid (non-segmented) sieve performs

$$O\left(\frac{n}{3}\sum_{p=5}^{\sqrt{n}}\frac{1}{p}\right)$$

bit operations—a constant-factor improvement ($n/3$ instead of $n/2$) while retaining the classical $\sum 1/p$ behavior.

3.3 The Classic Sieve-iZ Algorithm (SiZ)

To quantify the benefit of the $6x \pm 1$ reduction alone, the *Classic Sieve-iZ* targets moderate limits (e.g., up to 2^{32}). It maintains two bitmaps—each of size $n/6$ bits—and uses the Xp-Identity to mark composites.

Pseudocode: `SiZ`

Algorithm 2 Classic Sieve-iZ (SiZ)

```
procedure SiZ(int : n) → INT_ARRAY
Input: int: n                                          ▷ upper limit
Output: INT_ARRAY: primes                              ▷ array of integers containing primes up to n
                                                       ▷ 1. Initialization
    INT_ARRAY: primes ← {2,3}                          ▷ initialize primes array with the wheel base primes
    int: x_n ← ⌈(n+1)/6⌉                               ▷ iZ upper index limit
    BITMAP: x5 ← BITMAP_INIT(x_n, 1)                   ▷ create a bitmap of size x_n, and all bits set to 1
    BITMAP: x7 ← BITMAP_INIT(x_n, 1)                   ▷ create a bitmap of size x_n, and all bits set to 1
                                                       ▷ 2. Sieve: mark composites and collect primes
    int: n_sqrt ← ⌈√n⌉                                 ▷ root limit
    for int: x ← 1 to x_n do
        if BITMAP_GET_BIT(x5, x) then                  ▷ i.e., iZ- prime
            int: p ← iZ(x, −1)                         ▷ construct p from x using the iZ function
            INT_ARRAY_PUSH(primes, p)                  ▷ append p to primes
            if p < n_sqrt then                         ▷ i.e., root prime
                BITMAP_CLEAR_STEPS(x5, p, p·x + x, x_n)   ▷ marking composites of p in iℤ⁻
                BITMAP_CLEAR_STEPS(x7, p, p·x − x, x_n)   ▷ marking composites of p in iℤ⁺
            end if
        end if
        if BITMAP_GET_BIT(x7, x) then                  ▷ i.e., iZ+ prime, repeat for the other partition
            int: p ← iZ(x, 1)
            INT_ARRAY_PUSH(primes, p)
            if p < n_sqrt then
                BITMAP_CLEAR_STEPS(x5, p, p·x − x, x_n)
                BITMAP_CLEAR_STEPS(x7, p, p·x + x, x_n)
            end if
        end if
    end for
                                                       ▷ 3. Return the result
    return primes
end procedure
```

Space Complexity: Uses linear space $O(n/3)$.

Time Complexity: Additive marking within $i\mathbb{Z}$ yields bit-cost

$$O\left(\frac{n}{3}\sum_{p=5}^{\sqrt{n}}\frac{1}{p}\right),$$

which is still $O(n \log\log n)$, but with a constant-factor improvement over classic SoE.

Limitations. Linear workspace makes the solid sieve unsuitable for large n; segmentation remedies this.

4 The iZm Structure: Extended Wheel Optimization

Pseudocode: `SiZ`

Algorithm 3 Segmented Sieve-iZm (SiZm)

```
procedure SiZm(int : n) → INT_ARRAY
Input: int: n                                                   ▷ upper limit
Output: INT_ARRAY: primes                                       ▷ array of integers containing primes up to n
                                                                ▷ 1. Initialization
    int: x_n ← ⌈(n+1)/6⌉                                        ▷ iZ upper index limit
    int: n_sqrt ← ⌈√n⌉                                          ▷ root limit
    int: vx ← iZm_compute_limited_vx(n, 6)                      ▷ compute the segment length vx
    IZM: iZm ← iZm_init(vx, n_sqrt)
    INT_ARRAY: primes ← {}                                      ▷ initialize empty primes array
    int: k ← 0                                                  ▷ keeps track of how many pre-sieved primes
    while (6 · vx) % iZm.root_primes[k] == 0 do                 ▷ add pre-sieved primes (those dividing 6 · vx)
        int_array_push(primes, iZm.root_primes[k])              ▷ append pre-sieved prime to primes array
        k ← k + 1                                               ▷ update k, the count of pre-sieved primes to skip later
    end while
                                                                ▷ 2. Sieve vx segments for y in [0, y_limit]
    int: y_limit ← ⌊x_n/vx⌋                                     ▷ limit for y iterations
    int: x_limit ← vx                                           ▷ default x limit for full segments
    int: yvx ← 0                                                ▷ current base value (y × vx)
    for int: y ← 0 to y_limit (inclusive) do
                                                                ▷ a. Initialize active bitmaps from iZm base bitmaps
        BITMAP: x5 ← bitmap_clone(iZm.base_x5)                  ▷ bitmap for active sieve in iZm5
        BITMAP: x7 ← bitmap_clone(iZm.base_x7)                  ▷ bitmap for active sieve in iZm7
        if y == y_limit then
            x_limit ← x_n % vx                                  ▷ adjust x limit for last segment
        end if
                                                                ▷ b. Mark composites of root primes in current segment
        int: root_limit ← √(6 · (yvx + x_limit)) + 1            ▷ local root limit for current segment
        for int: i ← k to iZm.root_primes.count do              ▷ for each root prime, skipping the first k primes
            int: p ← iZm.root_primes[i]                         ▷ current root prime
            if p > root_limit then                              ▷ beyond local root limit
                break
            end if
                                                                ▷ mark composites of p in current segment using iZm xp solver
            bitmap_clear_steps(x5, p, iZm_solve_for_xp(−1, p, vx, y), x_limit)
            bitmap_clear_steps(x7, p, iZm_solve_for_xp(1, p, vx, y), x_limit)
        end for
                                                                ▷ c. Collect unmarked indices as primes
        for int: x ← 1 to x_limit do
            if bitmap_get_bit(x5, x) then                       ▷ i.e. iZ- prime
                int_array_push(primes, iZ(yvx + x, −1))
            end if
            if bitmap_get_bit(x7, x) then                       ▷ i.e. iZ+ prime
                int_array_push(primes, iZ(yvx + x, 1))
            end if
        end for
        yvx ← yvx + vx                                          ▷ advance base value for next segment
    end for
                                                                ▷ 3. Return the result
    return primes
end procedure
```

5 The VX_SEG Module: Large-Scale Hybrid Sieve Tools

6 Random Prime Generation Tools

7 Conclusions

A Algebraic Study on the iZ Function

Lemma A.1. *Let $m > 1 \in \mathbb{N}$ be a constant modulus. Let $\mathbb{I}$ be the set of residues coprime to m, closed under multiplication. Let $iZ(x \in \mathbb{N}, i \in \mathbb{I}) = mx + i$.*

Let $p = iZ(x_p, i_p)$ and $q = iZ(x_q, i_q)$, then their product $z = pq = iZ(x_z, i_z) \in i\mathbb{Z}$, where

$$x_z = mx_px_q + i_px_q + i_qx_p, \quad i_z = (i_pi_q) \in \mathbb{I}.$$

Consequently, this equivalence holds for each prime factor p that divides z:

$$p \mid z \quad \Longleftrightarrow \quad x_z \equiv (\frac{i_z}{i_p})x_p \pmod{p}.$$

Proof. Expanding the product $z = pq$ gives:

$$z = pq = (mx_p + i_p)(mx_q + i_q) = m^2x_px_q + mi_px_q + mi_qx_p + i_pi_q.$$

Grouping terms,

$$z = m(mx_px_q + i_px_q + i_qx_p) + i_pi_q = mx_z + i_z.$$

This shows that z can be expressed in the form $iZ(x_z, i_z)$, where

- The *product residue* $i_z \in \mathbb{I}$ is the product of the factors residues i_p and i_q, yielding:

$$i_z = \begin{cases} 1 & \text{if } i_p = i_q, \\ -1 & \text{if } i_p \neq i_q. \end{cases}$$

- The *product index* x_z can be expressed in terms of the factors p and q as:

$$x_z = mx_px_q + i_px_q + i_qx_p = px_q + i_qx_p = qx_p + i_px_q,$$

  such that x_z satisfies

$$x_z \equiv \begin{cases} i_qx_p \pmod{p}, \\ i_px_q \pmod{q}. \end{cases}$$

  From this equivalence, we derived the Xp-Identity, which pinpoints the composite indices of a given prime p in the X-arrays using additive steps of p.

□

B Data Structures

B.1 Bitmap Module

B.2 Integer Array Module

References

[1] `https://github.com/Zprime137/iZ-library`

[2] International Organization for Standardization. Programming languages – C ISO/IEC 9899:2018 International Organization for Standardization, Geneva, Switzerland, 2018.

[3] Ghidarcea, M., and Popescu, D. *Prime Number Sieving—A Systematic Review with Performance Analysis*. Algorithms (2024), 17, 157.

[4] Bays, C.; Hudson, R.H. The Segmented Sieve of Eratosthenes and Primes in Arithmetic Progressions to 1012. BIT 1977, 17, 121–127. [CrossRef]

[5] Paul Pritchard. Linear prime-number sieves: a family tree. *Science of Computer Programming*, 9(1):17–35, 1987.

[6] Leonhard Euler. Variae observationes circa series infinitas.

[7] A. O. L. Atkin and Daniel J. Bernstein. Prime sieves using binary quadratic forms. *Mathematics of Computation*, 73(246):1023–1030, 2004.

[8] Charters, B.A. Algorithm 311: Prime Number Generator 2. Commun. ACM 1967, 10, 570.

[9] Singleton, R.C. Algorithm 357: An Efficient Prime Number Generator [A1]. Commun. ACM 1969, 12, 563–564.